

The Idle Machine

What the World Spends Rebuilding What It Already Built

A measured kernel, an honest extrapolation, and a top-three-compiler scenario

The Flux Foundation

model computed by `flux-axiv-latex`, thermodynamic floor by `flux-science`,
related work drawn from a live arXiv sweep

July 26, 2026

216×

MEASURED — the redundant
work removed from
one real workspace

287,467

SCENARIO — engineer-
years returned per
year at top-three share

0.0132%

SCENARIO — that
same scenario's share
of ICT carbon: the
headline we refuse

Abstract

The world runs a machine that produces nothing: it rebuilds software that has not changed. This paper measures that waste stream in one fully instrumented workspace, then extrapolates it in four numbered rungs — every multiplier declared, printed and graded — to the scenario the title invites: a build orchestrator with top-three share of the world's compiled builds. The measured kernel is a single bookkeeping defect: units restored from a content cache skipped the compiler and therefore never wrote their fingerprint file, so **19** poisoned roots cascaded into **147** downstream units and confirming that nothing had changed cost **215.9 s** instead of **1.0 s** — a 216× removal of pure waste. Extrapolated at 20% share, using laptop-class power rather than the 720 W box we actually measured on, the recoverable stream is **0.204 TWh** and **97,856 t CO₂e** a year — and here the paper deflates its own headline: that is **0.0132%** of ICT's carbon footprint and **0.00070%** of world electricity. As a climate intervention it is noise. What is not noise is time: **287,467 engineer-years** returned every year, worth **\$50.2 bn** at the stated rate, and a correctness dividend on roughly **157,500** defect-carrying releases annually. We then apply the Jevons rebound honestly: at full rebound the energy saving vanishes entirely while the time and correctness dividends survive intact, which is why those are the claims we make. Two findings cut against our own interest. The thermodynamic floor for deciding “has anything changed?” across the whole scenario is about **one microjoule per year** for the entire planet — we spend $\sim 1.12 \times 10^{20} \times$ that — so the waste is coordination, not physics. And the single largest planetary lever is not a new compiler but a *default*: turning build caching on by default for the 70% of projects that do not use it recovers **0.609 TWh**, **3.0×** the entire top-three scenario.

1 Status of this claim

Read this before any number

This paper has a MEASURED kernel and a SCENARIO shell, and the two must not be confused.

Measured: one workspace, one instrument, dated. A no-op build of 215.9s repaired to 1.0s; the heaviest binary at 359s across 166 units re-declared dirty; a 36% unit-level cache hit rate afterwards.

Scenario: the orchestrator in question has, at the time of writing, no meaningful share of the world’s builds. It is a version-0.39 project maintained by one operator and a fleet of automated agents. “Top three” is a *multiplication we perform in public*, not a forecast, a roadmap, or a claim of adoption. Nothing in §5 should be read as a prediction.

Why publish it anyway: the unit economics — kWh, hours and tonnes per engineer-year of redundant build work — are share-independent. They are the reusable part. Any reader can replace our share, our population, or our power assumptions and recompute; the generator is a program, and §11 lists the six measurements that would refute us.

2 What “saving the world” can and cannot mean here

The ICT sector is responsible for roughly 2% of global carbon emissions [1], with data centres estimated at 1.1–1.5% of world electricity in the earlier literature [2, 3] and hyperscale facilities now measured directly at facility level [4]. Build and test compute is a slice of that slice. No compiler — however widely adopted, however efficient — is a climate intervention at the scale the phrase “save the world” implies, and this paper will not pretend otherwise. The field has already been burned once by exactly this move: highly visible language-energy rankings were causally misread by academics and industry leaders alike, a misreading that a careful causal model later had to undo [7].

What survives that deflation is still worth writing down. Three claims, in descending order of defensibility:

1. **There is a measurable waste stream** — machine time spent recomputing outputs that are already known — and it can be removed without asking a single human to change their behaviour. That is rare among sustainability interventions, which usually require behaviour change and therefore suffer rebound [11].
2. **The dominant recovered resource is human, not electrical.** At 20% share the scenario returns **287,467 engineer-years** a year against **97,856 t CO₂e** — and the carbon figure is 0.0132% of ICT’s footprint, i.e. indistinguishable from zero, while the time figure is 13.7% of the affected engineers’ working lives.
3. **The same instrument that removes the waste removes a correctness risk**, and software is infrastructure whose failures are expensive at civilisational scale [18]. This is the dividend we would defend hardest and monetise least.

3 The waste stream, measured

• **MEASURED** In one instrumented workspace, builds that should have been no-ops were not. Confirming that an unchanged crate was unchanged cost 215.9s. The heaviest binary cost 359s across 166 units re-declared dirty on every invocation. The cause was not compilation but bookkeeping: units restored from the content cache skipped the compiler, so the fingerprint

dependency file was never written, and the build tool marked 19 roots permanently stale, cascading into 147 downstream units. After the repair — capture, require and materialise that file on cache restore — the same no-op measured 1.0s and the unit-level cache hit rate settled at 36%.

Two properties make this the right kernel for a planetary estimate. First, the removed work is *provably useless*: its output was already known, which is why removing it changes nothing observable except the wait. Second, the failure mode is *structural, not local* — a dependency-tracking invariant, of exactly the kind the build-systems literature has been modelling for over a decade [16], and adjacent to the reasons build durations are predictable at all [15] and why CI failures dominate developer frustration [17]. Nothing about it is peculiar to this project’s language or domain, which is what licenses — carefully — the extrapolation that follows. On the machine the measurement was taken on, one redundant build dissipated 154,728 J.

4 The extrapolation ladder

Planetary numbers earn distrust because the multiplication is hidden. Here it is, one rung at a time, each with its own evidence grade. A reader who rejects rung 3 can stop at rung 2 and still keep everything above it.

Rung	What	Multiplier		Result	Grade
0	One redundant build, our workspace	—		214.9 s wasted, 154,728 J on the measured box	● MEASURED
1	One engineer-year	9,200 builds×0.40	no-change	14.3 kWh + 82.8 kWh CI = 97.1 kWh; 252 h	⊕ DERIVED
2	One 1,000-engineer organisation	×1,000		97 MWh; 251,876 h; \$23,928,178	⊕ DERIVED
3	Addressable world population	30 M devs	×0.35 compiled	10,500,000 engineers	△ ASSUMED
4	Top-three share	×20%		0.204 TWh; 97,856 t CO ₂ e; 287,467 engineer-years	◇ SCENARIO

The assumptions, stated once. △ ASSUMED An engineer invokes the inner loop 40 times a day over 230 working days, of which 40% are no-change invocations; a wait beyond ten seconds triggers a task switch with probability 0.35 costing 90 s of recovery; CI consumes 2.0 machine-hours per engineer per day at 240 W with a data-centre PUE of 1.5, of which 50% is redundant; there are 30 million professional developers worldwide and 35% of their work runs on ahead-of-time compiled or type-checked toolchains; the grid emits 480 gCO₂/kWh and consumes 1.8 L of water per kWh; an engineer costs \$95 an hour fully loaded.

One assumption is deliberately against us. The local-loop power figure is 65 W — laptop class — although the measurement itself was taken on a 720 W server. Using our own instrument’s power would multiply the local energy term by 11.1×. We discount our own measurement because most of the world’s inner loops do not run on 48-core machines, and because a scenario paper that inflates its own kernel deserves to be disbelieved.

5 The scenario: one, five, and twenty per cent

◇ SCENARIO “Top three” is read here as 20% of the world’s compiled build invocations. The one and five per cent rows are included because they are the only rows any real project reaches first, and because a claim that only works at the top of the table is not an argument.

Share	Engineers	TWh/yr	t CO ₂ e/yr	ML water	Households	Eng-years/yr	Value/yr
1%	105,000	0.010	4,893	18	2,912	14,373	\$2.5 bn
5%	525,000	0.051	24,464	92	14,562	71,867	\$12.6 bn
20%	2,100,000	0.204	97,856	367	58,247	287,467	\$50.2 bn

Now the deflation. At the top row the recovered electricity is 0.204 TWh — **0.00070%** of world electricity consumption — and the avoided carbon is 97,856 t CO₂e, or **0.0132%** of ICT’s estimated footprint and **0.000264%** of global emissions. Rounded to the precision anyone reports climate policy in, it is zero. Any paper that led with the tonnage would be innumerate, and the literature on software’s energy footprint is explicit that estimates at this remove need their methodology stated before their totals [5, 6].

What the same row says about people. 287,467 engineer-years returned annually, \$50.2 bn at the stated rate, or 13.7% of the working life of every engineer in scope. That is not a rounding error in anyone’s units. It is, at the top row, the equivalent of hiring a mid-sized nation’s software workforce and paying it nothing — and it arrives without a single behavioural request, which is the property that makes it unusual.

For completeness, the two figures a sustainability report would normally lead with: the same electricity carries **367 ML** of cooling water at the assumed 1.8 L/kWh, and corresponds to the annual consumption of **58,247** households at 3,500 kWh each. Both are included because omitting them would look like selection, and both are subject to the same deflation: they are small, and §6 may erase them entirely.

6 The rebound, applied to ourselves

Efficiency gains in computing are routinely re-spent. The Jevons paradox has been given an explicit thermodynamic reading for cloud workloads [8]; the AI environmental debate has been analysed precisely as a rebound argument [9]; measured ML training footprints keep rising *despite* per-unit efficiency gains, which is the rebound observed rather than theorised [10]; sustainable-HCI work argues rebound should be embraced in the model rather than apologised for afterwards [11]; and the materiality of the sector bounds what any efficiency story can deliver [12]. A build system that makes builds nearly free is a textbook candidate: cheaper builds invite more builds, larger matrices, more speculative CI.

⊕ DERIVED So we apply it to our own headline. Let ρ be the fraction of the recovered machine time immediately re-spent on additional build work.

ρ	Net TWh/yr	Net t CO ₂ e/yr	Eng-years/yr	Verdict
0%	0.204 TWh	97,856	287,467	both stand
50%	0.102 TWh	48,928	287,467	both stand
100%	0.000 TWh	0	287,467	energy claim gone; time claim intact

The asymmetry is the whole point. At $\rho = 1$ the energy dividend is exactly zero and the carbon claim disappears with it. The time dividend does not, because re-spending recovered *human* attention on building more software is not a leak in the model — it is the outcome the model exists to produce. Rebound destroys an energy argument and completes a productivity one. This is why the paper’s load-bearing claims are time and correctness, and why the tonnage appears only to be dismissed. A reader who believes $\rho = 1$ — a defensible position on this literature — loses nothing this paper actually argues.

7 The correctness dividend at scale

◇ SCENARIO The same discipline that found the redundant work — independent verification of what an instrument claims — has a second output. In the companion paper the measured

case was a storage layer that passed its entire unit suite while returning 0.4% of reads, a suite carrying ~ 0.001 bits of information about the property anyone cared about. Applying the same arithmetic at scenario scale: 20% share implies roughly 3,150,000 releases a year at a team size of 8 and 12 releases per team per year, of which 157,500 carry a defect at the assumed 5% rate. A suite with zero verification yield passes all of them; an independent correctness probe that catches the class 95% of the time stops 149,625.

We deliberately do not monetise that number. Escape costs for infrastructure software span orders of magnitude — from a wasted afternoon to the historical failures catalogued in the formal-methods literature [18] — and any single dollar figure would be the least defensible line in the paper. The flaky-test literature makes the mechanism concrete from the noise side: ambiguous signals interfere with automated assessment of changes at industrial scale [19]. A build system with top-three share would be the largest single deployment surface for that ambiguity in the world, which is an argument for humility about defaults, not a boast.

8 One microjoule per planet-year

The redundant fraction has an unusual property: because its output is already known, the only physically necessary work is the *decision* that nothing changed. Erasing a bit at $T = 300$ K costs at least $k_B T \ln 2 = 2.87 \times 10^{-21}$ J [31, 30]. Comparing one digest per unit across a 170-unit workspace is 43520 bits, so one such decision has a floor of 1.25×10^{-16} J. At 20% share the scenario performs 7,728,000,000 such decisions a year, for a total thermodynamic floor of **9.66×10^{-7} J** — about one microjoule, for the entire planet, for a year.

⊕ DERIVED The same decisions actually consume 1.08×10^{14} J of local-loop electricity: a factor of 1.12×10^{20} above the floor. The claim needs its scope stated precisely, or it is dishonest: this floor applies *only* to the redundant fraction, where nothing is compiled and no new information is produced. Compiling changed code has a vastly higher and quite legitimate floor. But that is the point — the waste stream is precisely the part of the bill that physics does not require, and it sits some twenty orders of magnitude above its own minimum. Whatever is expensive about building software, it is not the universe’s opinion of the work.

9 The largest lever is a default, not a compiler

A large-scale study of 513,384 CI builds across 1,279 projects found that only 30% adopt caching at all [13]. That is not a technology gap; it is a *configuration* gap, and it is the most consequential number in this paper.

◇ SCENARIO Apply our own CI term to the 70% of projects that do not cache, across the whole addressable population rather than any one tool’s share: **0.609 TWh** a year — **3.0×** the entire top-three scenario in §5, available today, from tools that already exist, with no adoption of anything we build.

We report this because it is the finding a self-interested paper would omit. The lesson generalises past caching: the planetary footprint of a build tool is set less by its peak performance than by what it does when nobody configures it. A tool’s defaults are its actual environmental policy. Green-software practice reaches the same conclusion from the field — the wins that survive are the ones wired into how the system runs by default rather than left to intention [25], alongside genuinely orthogonal levers such as carbon-aware scheduling and workload placement [26, 27], and the observation that ordinary performance bugs carry a carbon cost of their own [24].

10 What a top-three compiler would owe

If the scenario in §5 were ever real, the interesting consequence would not be the savings. It would be that the tool had become infrastructure, and infrastructure acquires duties. Stated as commitments a reader could later hold us to:

1. **Determinism before speed.** Every performance claim paired with an independent correctness check on the same artifact. A cache that reports its own hit rate is not evidence; this project has the scar to prove it.
2. **Cache on by default, and provably free when nothing changed.** The 70% non-adoption figure is a design failure upstream of every user who did not configure it.
3. **Signed, reproducible artifacts.** A build tool at scale is a supply-chain choke point; pipeline poisoning is a demonstrated attack class [20], end-to-end reproducibility is achievable [23], and signing’s costs are exactly why adoption stalls [22, 21]. At top-three share those costs stop being optional.
4. **No telemetry rent.** Measurement of the tool must not require surveillance of its users. Everything in this paper was measured on the authors’ own machines.
5. **Published measurement, including the failures.** The instrument that lied is part of the record, not an embarrassment to be pruned from it.
6. **Energy reporting with its methodology attached.** Per the state of the art, a total without a stated method is not a measurement [5, 6] — and LLM-assisted development is now itself a term in the same budget [28], as is the cost of automated repair [29].

11 What would have to be true

Six measurements would move this paper’s numbers, and four of them could refute it outright. Each is stated as a gate someone could actually run.

#	Gate	Effect if it fails
1	The no-change fraction. We assume 40% of inner-loop invocations change nothing. Measure it on a real fleet by logging build inputs.	Linear: the entire local term scales with it.
2	Per-build energy on real hardware. Ours is one 720 W box, discounted to 65 W by assumption. Measure at the wall across representative laptops and CI runners [6].	Linear, either direction; could move the energy claim by 10 $\times$.
3	The redundant CI share. We assume 50% of CI machine-time is recomputation. Instrument a large CI estate.	CI is the larger of the two energy terms.
4	Generality of the defect. Our kernel is one dependency-tracking bug. Sample other toolchains for equivalent no-op inflation.	If it is peculiar to us, rungs 3–4 collapse entirely.
5	The rebound coefficient. We report $\rho \in \{0, 0.5, 1\}$ rather than estimating it. Measure build volume before and after a large speedup.	At $\rho = 1$ the energy claim is void by construction (§6).
6	Population and share. 30 M developers, 35% compiled toolchains, 20% share are all assumptions with no instrument behind them.	Rung 3 and 4 are scenario-grade for exactly this reason.

What we do not claim. Not that this is a climate intervention — §5 computes it as noise. Not that the scenario share is achievable. Not that redundant build work is the largest waste stream in computing; training and inference are plainly larger and are being measured by others [10, 9]. Not that removing waste is the same as building well. The claim is narrower and, we think, sturdier: there is a quantity of pure recomputation in the world, it is measurable from one honest instrument outward, most of it is a configuration failure rather than a technical limit, and the resource it wastes that no one gets back is time.

12 Related work

Retrieved by a live arXiv query at generation time; each entry is glossed with the opening claim of its own abstract, unedited.

- [1] *Carbon-Efficient Software Design and Development: A Systematic Literature Review* — The ICT sector, responsible for 2% of global carbon emissions, is under scrutiny calling for methodologies and tools to design and develop software in an environmentally sustainable-by-design manner.
- [2] *Take a break: cloud scheduling optimized for real-time electricity pricing* — Cloud computing revolutionised the industry with its elastic, on-demand approach to computational resources, but has lead to a tremendous impact on the environment.

- [3] *Data Center-Enabled High Altitude Platforms: A Green Computing Alternative* — Information technology organizations and companies are seeking greener alternatives to traditional terrestrial data centers to mitigate global warming and reduce carbon emissions.
- [4] *Assessing the Carbon Emissions and Energy Consumption of U.S. Hyperscale Data Centers* — The rapid proliferation of hyperscale data centers in the US, mainly driven by the adoption of artificial intelligence, has raised concerns about this industry’s environmental footprint.
- [5] *Calculating Software’s Energy Use and Carbon Emissions: A Survey of the State of Art, Challenges, and the Way Ahead* — The proliferation of software and AI comes with a hidden risk: its growing energy and carbon footprint.
- [6] *Estimating the Energy Footprint of Software Systems: a Primer* — In Green Software Development, quantifying the energy footprint of a software system is one of the most basic activities.
- [7] *It’s Not Easy Being Green: On the Energy Efficiency of Programming Languages* — Does the choice of programming language affect energy consumption? Previous highly visible studies have established associations between certain programming languages and energy consumption.
- [8] *The Jevons Paradox In Cloud Computing: A Thermodynamics Perspective* — How do we explain the simultaneous growth in energy efficiency of cloud computing and its energy consumption? The Jevons paradox provides one perspective of this phenomenon.
- [9] *From Efficiency Gains to Rebound Effects: The Problem of Jevons’ Paradox in AI’s Polarized Environmental Debate* — As the climate crisis deepens, artificial intelligence has emerged as a contested force: some champion its potential to advance renewable energy, materials discovery, and large-scale emissions monitor.
- [10] *The Environmental Impacts of Machine Learning Training Keep Rising Evidencing Rebound Effect* — Recent Machine Learning approaches have shown increased performance on benchmarks but at the cost of escalating computational demands.
- [11] *How Viable are Energy Savings in Smart Homes? A Call to Embrace Rebound Effects in Sustainable HCI* — As part of global climate action, digital technologies are seen as a key enabler of energy efficiency savings.
- [12] *The Case for Time-Shared Computing Resources* — The environmental impact of Information and Communication Technologies continues to grow, driven notably by increasing usage, rebound effects, and emerging demands.
- [13] *The Promise and Reality of Continuous Integration Caching: An Empirical Study of Travis CI Builds* — Continuous Integration provides early feedback by automatically building software, but long build durations can hinder developer productivity.
- [14] *The Cost of Downgrading Build Systems: A Case Study of Kubernetes* — Since developers invoke the build system frequently, its performance can impact productivity.
- [15] *Built to Last or Built Too Fast? Evaluating Prediction Models for Build Times* — Automated builds are integral to Continuous Integration.
- [16] *A model and framework for reliable build systems* — Reliable and fast builds are essential for rapid turnaround during development and testing.
- [17] *Practitioners’ Challenges and Perceptions of CI Build Failure Predictions at Atlassian* — Continuous Integration build failures could significantly impact the software development

process and teams, such as delaying the release of new features and reducing developers' productivity.

- [18] *Software is infrastructure: failures, successes, costs, and the case for formal verification* — In this chapter we outline the role that software has in modern society, along with the staggering costs of poor software quality.
- [19] *Flaky Tests in a Large Industrial Database Management System: An Empirical Study of Fixed Issue Reports for SAP HANA* — Flaky tests yield different results when executed multiple times for the same version of the source code.
- [20] *Build Code is Still Code: Finding the Antidote for Pipeline Poisoning* — Open source C code underpins society's computing infrastructure.
- [21] *Analyzing Challenges in Deployment of the SLSA Framework for Software Supply Chain Security* — In 2023, Sonatype reported a 200% increase in software supply chain attacks, including major build infrastructure attacks.
- [22] *Why Software Signing (Still) Matters: Trust Boundaries in the Software Supply Chain* — Software signing provides a formal mechanism for provenance by ensuring artifact integrity and verifying producer identity.
- [23] *Building a Secure Software Supply Chain with GNU Guix* — The software supply chain designates the series of steps taken to go from source code published by developers to executables running on users' computers.
- [24] *Profiling the carbon footprint of performance bugs* — Much debate nowadays is devoted to the impacts of modern information and communication technology on global carbon emissions.
- [25] *Putting green software principles into practice* — The need and theoretical methods for measuring and reducing CO2 emitted by computing systems are well understood, but real-world examples are still limited.
- [26] *Carbon-aware Software Services* — The significant carbon footprint of the ICT sector calls for methodologies to contain carbon emissions of running software.
- [27] *On the Effectiveness of the 'Follow-the-Sun' Strategy in Mitigating the Carbon Footprint of AI in Cloud Instances* — Follow-the-Sun is a theoretical computational model aimed at minimizing the carbon footprint of computer workloads.
- [28] *Comparative Analysis of Carbon Footprint in Manual vs. LLM-Assisted Code Development* — Large Language Models have significantly transformed software development.
- [29] *Energy Consumption of Automated Program Repair* — Automated program repair aims to automatize the process of repairing software bugs in order to reduce the cost of maintaining software programs.
- [30] *Landauer-Limited Dissipation in Quantum-Flux-Parametron Logic* — The Landauer limit is to irreversible logic what the Carnot cycle is to heat engines.
- [31] *Reset and switch protocols at Landauer limit in a graphene buckled ribbon* — Heat produced during a reset operation is meant to show a fundamental bound known as the Landauer limit.

Sources — the measured record behind the kernel

- *The Legibility Dividend: An Executive Model of Verifiable Software Production*, v0 (2026-07-26) — the immediate predecessor: the same kernel priced for one organisation, and the source of the

Verification Information Yield instrument used in §7. https://quillon.xyz/downloads/SIGIL_LEGIBILITY_DIVIDEND_v0.pdf.

- *The Measurement Book: Laws of the SIGIL Graph — the measured record*, v0 (2026-07-25) — ARC-15 is the build-latency arc measured in §3; its Rule 0 is why this paper states which rung every number sits on. https://quillon.xyz/downloads/SIGIL_MEASUREMENT_BOOK_v0.pdf.
- *The SIGIL Failure Atlas*, v0 (2026-07-23) — the incident taxonomy, including the measurement class that contains the instrument which lied. https://quillon.xyz/downloads/SIGIL_FAILURE_ATLAS_v0.pdf.
- *The Thermodynamic Ledger*, v0 (2026-07-24) — the first paper in this computed-at-generation-time series; the Landauer treatment in §8 follows its method. https://quillon.xyz/downloads/SIGIL_THERMODYNAMIC_LEDGER_v0.pdf.
- *What the Chain Knows — The Philosophy of the SIGIL Graph*, v0.3 (2026-07-22) — the evidence-grading discipline this paper applies to its own extrapolation. https://quillon.xyz/downloads/SIGIL_PHILOSOPHY_v0.pdf.

13 Reproducibility

This document is the output of a Rust binary in the Flux tree (`flux-arxiv-latex/src/bin/idle_machine.rs`), built and run through the `fluxc` orchestrator. Every figure above is computed at generation time; the measured constants and the assumed multipliers are declared once, at the top of the program, as named variables in two clearly separated blocks. The Landauer floor uses `flux-science`'s CODATA Boltzmann constant ($k_B = 1.38 \times 10^{-23}$ J/K). The bibliography is generated from a live arXiv sweep parsed by the same crate. To disagree with this paper, change a variable and recompile: the argument is a program, and its conclusions are functions of inputs you can see. That is the only form in which a planetary extrapolation deserves to be published.

14 Coda: the machine that produces nothing

There is something absurd, and worth sitting with, about the scale of what has been described here. Every day, a very large number of extremely fast machines are asked a question to which the answer is already recorded, and they answer it slowly, by redoing the work. Physics charges about a microjoule a year for the whole planet's worth of that question. We pay it in gigawatt-hours and, far more expensively, in the attention of the people waiting.

The temptation with a number like that is to make it a crusade. We have tried instead to make it legible, because the honest version is more useful than the heroic one: the carbon is noise, the rebound may eat the electricity entirely, and the largest available lever is not our compiler but a checkbox that most projects never tick. What is left when all of that is subtracted is still a real thing — hundreds of thousands of engineer-years a year of human life spent watching a progress bar recompute a known answer, and a class of verification failure that the same discipline happens to catch on the way past.

A compiler will not save the world. But a tool used by a large fraction of the world has one obligation it cannot delegate: to not waste what it is given. That is a smaller promise than the title of this paper implies, and it is the one we can keep — and, unusually for a promise, one whose keeping is measurable by anyone who cares to rerun the program.

References

- [1] Ornela Danushi, Stefano Forti, Jacopo Soldani: *Carbon-Efficient Software Design and Development: A Systematic Literature Review*. arXiv:2407.19901 (2024). <https://arxiv.org/abs/2407.19901>
- [2] Drazen Lucanin, Ivona Brandic: *Take a break: cloud scheduling optimized for real-time electricity pricing*. arXiv:1307.7037 (2013). <https://arxiv.org/abs/1307.7037>
- [3] Wiem Abderrahim, Osama Amin, Basem Shihada: *Data Center-Enabled High Altitude Platforms: A Green Computing Alternative*. arXiv:2309.09241 (2023). <https://arxiv.org/abs/2309.09241>

- [4] Gianluca Guidi, Francesca Dominici, Tiziano Squartini, et al.: *Assessing the Carbon Emissions and Energy Consumption of U.S. Hyperscale Data Centers*. arXiv:2606.05420 (2026). <https://arxiv.org/abs/2606.05420>
- [5] Priyavanshi Pathania, Nikhil Bamby, Rohit Mehra, et al.: *Calculating Software’s Energy Use and Carbon Emissions: A Survey of the State of Art, Challenges, and the Way Ahead*. arXiv:2506.09683 (2025). <https://arxiv.org/abs/2506.09683>
- [6] Fernando Castor: *Estimating the Energy Footprint of Software Systems: a Primer*. arXiv:2407.11611 (2024). <https://arxiv.org/abs/2407.11611>
- [7] Nicolas van Kempen, Hyuk-Je Kwon, Dung Tuan Nguyen, et al.: *It’s Not Easy Being Green: On the Energy Efficiency of Programming Languages*. arXiv:2410.05460 (2024). <https://arxiv.org/abs/2410.05460>
- [8] Prateek Sharma: *The Jevons Paradox In Cloud Computing: A Thermodynamics Perspective*. arXiv:2411.11540 (2024). <https://arxiv.org/abs/2411.11540>
- [9] Alexandra Sasha Luccioni, Emma Strubell, Kate Crawford: *From Efficiency Gains to Rebound Effects: The Problem of Jevons’ Paradox in AI’s Polarized Environmental Debate*. arXiv:2501.16548 (2025). <https://arxiv.org/abs/2501.16548>
- [10] Clement Morand, Anne-Laure Ligozat, Aurelie Neveol: *The Environmental Impacts of Machine Learning Training Keep Rising Evidencing Rebound Effect*. arXiv:2510.09022 (2025). <https://arxiv.org/abs/2510.09022>
- [11] Christina Bremer, Harshit Gujral, Michelle Lin, et al.: *How Viable are Energy Savings in Smart Homes? A Call to Embrace Rebound Effects in Sustainable HCI*. arXiv:2506.14653 (2025). <https://arxiv.org/abs/2506.14653>
- [12] Pierre Jacquet, Adrien Luxey-Bitri: *The Case for Time-Shared Computing Resources*. arXiv:2507.19287 (2025). <https://arxiv.org/abs/2507.19287>
- [13] Taher A. Ghaleb, Daniel Alencar da Costa, Ying Zou: *The Promise and Reality of Continuous Integration Caching: An Empirical Study of Travis CI Builds*. arXiv:2601.19146 (2026). <https://arxiv.org/abs/2601.19146>
- [14] Gareema Ranjan, Mahmoud Alfadel, Gengyi Sun, et al.: *The Cost of Downgrading Build Systems: A Case Study of Kubernetes*. arXiv:2510.20041 (2025). <https://arxiv.org/abs/2510.20041>
- [15] Ekaba Bisong, Eric Tran, Olga Baysal: *Built to Last or Built Too Fast? Evaluating Prediction Models for Build Times*. arXiv:1712.06796 (2017). <https://arxiv.org/abs/1712.06796>
- [16] Derrick Coetzee, Anand Bhaskar, George Necula: *A model and framework for reliable build systems*. arXiv:1203.2704 (2012). <https://arxiv.org/abs/1203.2704>
- [17] Yang Hong, Chakkrit Tantithamthavorn, Jirat Pasuksmit, et al.: *Practitioners’ Challenges and Perceptions of CI Build Failure Predictions at Atlassian*. arXiv:2402.09651 (2024). <https://arxiv.org/abs/2402.09651>
- [18] Giovanni Bernardi, Adrian Francalanza, Marco Peressotti, et al.: *Software is infrastructure: failures, successes, costs, and the case for formal verification*. arXiv:2506.13821 (2025). <https://arxiv.org/abs/2506.13821>
- [19] Alexander Berndt, Thomas Bach, Sebastian Baltes: *Flaky Tests in a Large Industrial Database Management System: An Empirical Study of Fixed Issue Reports for SAP HANA*. arXiv:2602.03556 (2026). <https://arxiv.org/abs/2602.03556>
- [20] Brent Pappas, Paul Gazzillo: *Build Code is Still Code: Finding the Antidote for Pipeline Poisoning*. arXiv:2601.08995 (2026). <https://arxiv.org/abs/2601.08995>
- [21] Mahzabin Tamanna, Sivana Hamer, Mindy Tran, et al.: *Analyzing Challenges in Deployment of the SLSA Framework for Software Supply Chain Security*. arXiv:2409.05014 (2024). <https://arxiv.org/abs/2409.05014>
- [22] Kelechi G. Kalu, James C. Davis: *Why Software Signing (Still) Matters: Trust Boundaries in the Software Supply Chain*. arXiv:2510.04964 (2025). <https://arxiv.org/abs/2510.04964>

- [23] Ludovic Courtes: *Building a Secure Software Supply Chain with GNU Guix*. arXiv:2206.14606 (2022). <https://arxiv.org/abs/2206.14606>
- [24] Iztok Fister, Dusan Fister, Vili Podgorelec, et al.: *Profiling the carbon footprint of performance bugs*. arXiv:2401.01782 (2024). <https://arxiv.org/abs/2401.01782>
- [25] James Uther: *Putting green software principles into practice*. arXiv:2601.09741 (2026). <https://arxiv.org/abs/2601.09741>
- [26] Stefano Forti, Jacopo Soldani, Antonio Brogi: *Carbon-aware Software Services*. arXiv:2405.12582 (2024). <https://arxiv.org/abs/2405.12582>
- [27] Roberto Vergallo, Luis Cruz, Alessio Errico, et al.: *On the Effectiveness of the 'Follow-the-Sun' Strategy in Mitigating the Carbon Footprint of AI in Cloud Instances*. arXiv:2506.10990 (2025). <https://arxiv.org/abs/2506.10990>
- [28] Kuen Sum Cheung, Mayuri Kaul, Gunel Jahangirova, et al.: *Comparative Analysis of Carbon Footprint in Manual vs. LLM-Assisted Code Development*. arXiv:2505.04521 (2025). <https://arxiv.org/abs/2505.04521>
- [29] Matias Martinez, Silverio Martinez-Fernandez, Xavier Franch: *Energy Consumption of Automated Program Repair*. arXiv:2211.12104 (2022). <https://arxiv.org/abs/2211.12104>
- [30] Quentin Herr: *Landauer-Limited Dissipation in Quantum-Flux-Parametron Logic*. arXiv:2504.04284 (2025). <https://arxiv.org/abs/2504.04284>
- [31] I. Neri, M. Lopez-Suarez, D. Chiuchiu, et al.: *Reset and switch protocols at Landauer limit in a graphene buckled ribbon*. arXiv:1507.07450 (2015). <https://arxiv.org/abs/1507.07450>